

SQL Practice Questions with Answers

Patient Visit Records Dataset — Basic to Advanced, Normalization & Decomposition
Data Analyst
August 2026

[SQL Practice Questions with Answers](#)
[Section 1 — Basic Level \(Retrieval & Filtering\)](#)
[Section 2 — Intermediate Level \(Aggregation, Grouping, Conditional Logic\)](#)
[Section 3 — Advanced Level \(Joins, Subqueries, CTEs, Window Functions\)](#)
[Section 4 — Normalization \(Conceptual & Applied\)](#)
[Section 5 — Decomposition \(Schema Design & Migration SQL\)](#)

SQL Practice Questions with Answers

Dataset: new_data_cleaned_excel.xlsx → assume loaded into table **patient_visits** **Scope:** ~100,000 rows | 39 columns | Healthcare — patient demographics, diagnosis, treatment, outcomes

Dataset Reference

Category	Columns
Patient Info	Patient_ID, Age, Gender, State, Socioeconomic_Status, Occupation
Visit Info	Visit_Date, Visit_Year, Visit_Month, Age_Group
Clinical	Symptoms, Primary_Diagnosis, Fasting_Blood_Glucose_mg_dL, Glycemic_Status, HbA1c_mmol_mol, Total_Cholesterol_mg_dL, BMI_kg_meter_square, BMI_Status
Treatment	Treatment_Type, Treatment_Outcome
Diagnostics	Imaging_Type, Imaging_Findings
Facility	Hospital_Type, Insurance_Covered
Symptom Flags	Symptom_Blurred_Vision ... Symptom_Weight_Loss (14 one-hot columns)

Section 1 — Basic Level (Retrieval & Filtering)

Q1. Retrieve all columns for the first 10 patient visit records.

```
SELECT *
FROM patient_visits
LIMIT 10;
```

Q2. List distinct values of Primary_Diagnosis.

```
SELECT DISTINCT Primary_Diagnosis
FROM patient_visits;
```

Q3. Find all visits where Hospital_Type is 'Government'.

```
SELECT *
FROM patient_visits
WHERE Hospital_Type = 'Government';
```

Q4. Retrieve Patient_ID, Age, Gender for patients aged above 60.

```
SELECT Patient_ID, Age, Gender
FROM patient_visits
WHERE Age > 60;
```

Q5. List all visits from 'West Bengal', sorted by Visit_Date descending.

```
SELECT *
FROM patient_visits
WHERE State = 'West Bengal'
ORDER BY Visit_Date DESC;
```

Q6. Find all records where Insurance_Covered is 'Not Covered'.

```
SELECT *
FROM patient_visits
WHERE Insurance_Covered = 'Not Covered';
```

Q7. Retrieve all visits that occurred in the year 2025.

```
SELECT *
FROM patient_visits
WHERE Visit_Year = 2025;
```

Q8. Find patients whose BMI_Status is 'Obesity'.

```
SELECT *
FROM patient_visits
WHERE BMI_Status = 'Obesity';
```

Q9. List all distinct Treatment_Type values.

```
SELECT DISTINCT Treatment_Type
FROM patient_visits;
```

Q10. Retrieve all visits where Treatment_Outcome is 'Recovered' or 'Improved'.

```
SELECT *
FROM patient_visits
WHERE Treatment_Outcome IN ('Recovered', 'Improved');
```

Section 2 — Intermediate Level (Aggregation, Grouping, Conditional Logic)

Q11. Count total visits per State.

```
SELECT State, COUNT(*) AS total_visits
FROM patient_visits
GROUP BY State
ORDER BY total_visits DESC;
```

Q12. Average Age of patients per Primary_Diagnosis.

```
SELECT Primary_Diagnosis, ROUND(AVG(Age), 1) AS avg_age
FROM patient_visits
GROUP BY Primary_Diagnosis
ORDER BY avg_age DESC;
```

Q13. Number of patients per Gender, for each Hospital_Type.

```
SELECT Hospital_Type, Gender, COUNT(*) AS patient_count
FROM patient_visits
GROUP BY Hospital_Type, Gender
ORDER BY Hospital_Type, Gender;
```

Q14. Most common Primary_Diagnosis overall.

```
SELECT Primary_Diagnosis, COUNT(*) AS visit_count
FROM patient_visits
GROUP BY Primary_Diagnosis
ORDER BY visit_count DESC
LIMIT 1;
```

Q15. Average Total_Cholesterol_mg_dL grouped by BMI_Status.

```
SELECT BMI_Status, ROUND(AVG(Total_Cholesterol_mg_dL), 1) AS avg_cholesterol
FROM patient_visits
GROUP BY BMI_Status
ORDER BY avg_cholesterol DESC;
```

Q16. Diagnoses with more than 5,000 recorded visits.

```
SELECT Primary_Diagnosis, COUNT(*) AS visit_count
FROM patient_visits
GROUP BY Primary_Diagnosis
HAVING COUNT(*) > 5000
ORDER BY visit_count DESC;
```

Q17. Classify patients into age bands using CASE WHEN and count each group.

```
SELECT
CASE
```

```

        WHEN Age < 18 THEN 'Minor'
        WHEN Age BETWEEN 18 AND 59 THEN 'Adult'
        ELSE 'Senior'
    END AS age_band,
    COUNT(*) AS patient_count
FROM patient_visits
GROUP BY
    CASE
        WHEN Age < 18 THEN 'Minor'
        WHEN Age BETWEEN 18 AND 59 THEN 'Adult'
        ELSE 'Senior'
    END;

```

Q18. Percentage of visits per Treatment_Outcome.

```

SELECT
    Treatment_Outcome,
    COUNT(*) AS outcome_count,
    ROUND(100.0 * COUNT(*) / SUM(COUNT(*) OVER ()), 2) AS pct_of_total
FROM patient_visits
GROUP BY Treatment_Outcome
ORDER BY pct_of_total DESC;

```

Q19. Occupation with the highest average HbA1c_mmol_mol.

```

SELECT Occupation, ROUND(AVG(HbA1c_mmol_mol), 2) AS avg_hb1c
FROM patient_visits
GROUP BY Occupation
ORDER BY avg_hb1c DESC
LIMIT 1;

```

Q20. Per State, count of Covered vs Not Covered visits.

```

SELECT
    State,
    SUM(CASE WHEN Insurance_Covered = 'Covered' THEN 1 ELSE 0 END) AS covered,
    SUM(CASE WHEN Insurance_Covered = 'Not Covered' THEN 1 ELSE 0 END) AS not_covered
FROM patient_visits
GROUP BY State
ORDER BY State;

```

Q21. Month with the highest number of visits, per year.

```

SELECT Visit_Year, Visit_Month, COUNT(*) AS visit_count
FROM patient_visits
GROUP BY Visit_Year, Visit_Month
QUALIFY RANK() OVER (PARTITION BY Visit_Year ORDER BY COUNT(*) DESC) = 1;

-- Portable version (no QUALIFY support, e.g. MySQL/Postgres < 15 without QUALIFY):
WITH monthly AS (
    SELECT Visit_Year, Visit_Month, COUNT(*) AS visit_count,
           RANK() OVER (PARTITION BY Visit_Year ORDER BY COUNT(*) DESC) AS rnk
    FROM patient_visits
    GROUP BY Visit_Year, Visit_Month
)
SELECT Visit_Year, Visit_Month, visit_count
FROM monthly
WHERE rnk = 1;

```

Q22. Top 5 diagnoses by average Fasting_Blood_Glucose__mg_dL.

```

SELECT Primary_Diagnosis, ROUND(AVG(Fasting_Blood_Glucose__mg_dL), 1) AS avg_glucose
FROM patient_visits
GROUP BY Primary_Diagnosis
ORDER BY avg_glucose DESC
LIMIT 5;

```

Section 3 — Advanced Level (Joins, Subqueries, CTEs, Window Functions)

These queries assume the normalized schema from Section 6 (patients, visits, diagnoses, symptoms, visit_symptoms, treatments, hospitals) unless noted otherwise.

Q23. Top diagnosis per state (CTE).

```

WITH diagnosis_counts AS (
    SELECT p.State, d.Diagnosis_Name, COUNT(*) AS cnt,
           RANK() OVER (PARTITION BY p.State ORDER BY COUNT(*) DESC) AS rnk
    FROM visits v
    JOIN patients p ON v.Patient_ID = p.Patient_ID
    JOIN diagnoses d ON v.Diagnosis_ID = d.Diagnosis_ID
    GROUP BY p.State, d.Diagnosis_Name
)
SELECT State, Diagnosis_Name, cnt

```

```
FROM diagnosis_counts
WHERE rnk = 1
ORDER BY State;
```

Q24. Rank patients within each diagnosis by HbA1c_mmol_mol (window function).

```
SELECT
    v.Visit_ID, v.Patient_ID, d.Diagnosis_Name, v.HbA1c_mmol_mol,
    RANK() OVER (PARTITION BY d.Diagnosis_Name ORDER BY v.HbA1c_mmol_mol DESC) AS hba1c_rank
FROM visits v
JOIN diagnoses d ON v.Diagnosis_ID = d.Diagnosis_ID;
```

Q25. Running total of visits per month (window function).

```
WITH monthly AS (
    SELECT Visit_Year, Visit_Month, COUNT(*) AS visit_count
    FROM patient_visits
    GROUP BY Visit_Year, Visit_Month
)
SELECT
    Visit_Year, Visit_Month, visit_count,
    SUM(visit_count) OVER (ORDER BY Visit_Year, Visit_Month
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total
FROM monthly
ORDER BY Visit_Year, Visit_Month;
```

Q26. Compare Treatment_Outcome to previous visit (LAG).

```
SELECT
    Visit_ID, Patient_ID, Visit_Date, Treatment_Outcome,
    LAG(Treatment_Outcome) OVER (PARTITION BY Patient_ID ORDER BY Visit_Date) AS previous_outcome
FROM visits
QUALIFY previous_outcome IS NOT NULL;

-- Portable version:
WITH ranked AS (
    SELECT
        Visit_ID, Patient_ID, Visit_Date, Treatment_Outcome,
        LAG(Treatment_Outcome) OVER (PARTITION BY Patient_ID ORDER BY Visit_Date) AS previous_outcome
    FROM visits
)
SELECT * FROM ranked WHERE previous_outcome IS NOT NULL;
```

Q27. Patients above the average cholesterol for their own age group (correlated subquery).

```
SELECT pv.Patient_ID, pv.Age_Group, pv.Total_Cholesterol_mg_dL
FROM patient_visits pv
WHERE pv.Total_Cholesterol_mg_dL > (
    SELECT AVG(pv2.Total_Cholesterol_mg_dL)
    FROM patient_visits pv2
    WHERE pv2.Age_Group = pv.Age_Group
);
```

Q28. Patients who visited more than one Hospital_Type (self-join / EXISTS).

```
SELECT DISTINCT v1.Patient_ID
FROM visits v1
JOIN hospitals h1 ON v1.Hospital_ID = h1.Hospital_ID
WHERE EXISTS (
    SELECT 1
    FROM visits v2
    JOIN hospitals h2 ON v2.Hospital_ID = h2.Hospital_ID
    WHERE v2.Patient_ID = v1.Patient_ID
        AND h2.Hospital_Type <> h1.Hospital_Type
);
```

Q29. Visit details with comma-separated symptom list (JOIN + STRING_AGG).

```
SELECT
    v.Visit_ID, p.Patient_ID, p.Age, p.Gender, d.Diagnosis_Name,
    STRING_AGG(s.Symptom_Name, ', ') AS symptoms
    -- MySQL: GROUP_CONCAT(s.Symptom_Name SEPARATOR ', ')
FROM visits v
JOIN patients p ON v.Patient_ID = p.Patient_ID
JOIN diagnoses d ON v.Diagnosis_ID = d.Diagnosis_ID
JOIN visit_symptoms vs ON v.Visit_ID = vs.Visit_ID
JOIN symptoms s ON vs.Symptom_ID = s.Symptom_ID
GROUP BY v.Visit_ID, p.Patient_ID, p.Age, p.Gender, d.Diagnosis_Name;
```

Q30. Top 3 symptoms per diagnosis (CTE + window function).

```
WITH symptom_counts AS (
    SELECT d.Diagnosis_Name, s.Symptom_Name, COUNT(*) AS cnt,
    RANK() OVER (PARTITION BY d.Diagnosis_Name ORDER BY COUNT(*) DESC) AS rnk
FROM visits v
JOIN diagnoses d ON v.Diagnosis_ID = d.Diagnosis_ID
```

```

JOIN visit_symptoms vs ON v.Visit_ID = vs.Visit_ID
JOIN symptoms s      ON vs.Symptom_ID = s.Symptom_ID
GROUP BY d.Diagnosis_Name, s.Symptom_Name
)
SELECT Diagnosis_Name, Symptom_Name, cnt
FROM symptom_counts
WHERE rnk <= 3
ORDER BY Diagnosis_Name, rnk;

```

Q31. Month-over-month percentage change in visit counts (LAG).

```

WITH monthly AS (
  SELECT Visit_Year, Visit_Month, COUNT(*) AS visit_count
  FROM patient_visits
  GROUP BY Visit_Year, Visit_Month
)
SELECT
  Visit_Year, Visit_Month, visit_count,
  LAG(visit_count) OVER (ORDER BY Visit_Year, Visit_Month) AS prev_month_count,
  ROUND(
    100.0 * (visit_count - LAG(visit_count) OVER (ORDER BY Visit_Year, Visit_Month))
    / NULLIF(LAG(visit_count) OVER (ORDER BY Visit_Year, Visit_Month), 0), 2
  ) AS mom_pct_change
FROM monthly
ORDER BY Visit_Year, Visit_Month;

```

Q32. Patients whose BMI_Status changed between consecutive visits (self-join).

```

WITH ordered_visits AS (
  SELECT
    Patient_ID, Visit_Date, BMI_Status,
    LEAD(BMI_Status) OVER (PARTITION BY Patient_ID ORDER BY Visit_Date) AS next_bmi_status
  FROM visits
)
SELECT Patient_ID, Visit_Date, BMI_Status, next_bmi_status
FROM ordered_visits
WHERE next_bmi_status IS NOT NULL
  AND next_bmi_status <> BMI_Status;

```

Q33. Pivot Treatment_Outcome counts per State (CASE WHEN + SUM).

```

SELECT
  State,
  SUM(CASE WHEN Treatment_Outcome = 'Recovered' THEN 1 ELSE 0 END) AS recovered,
  SUM(CASE WHEN Treatment_Outcome = 'Improved'   THEN 1 ELSE 0 END) AS improved,
  SUM(CASE WHEN Treatment_Outcome = 'Stable'      THEN 1 ELSE 0 END) AS stable,
  SUM(CASE WHEN Treatment_Outcome = 'Referred'    THEN 1 ELSE 0 END) AS referred,
  SUM(CASE WHEN Treatment_Outcome = 'Worsened'    THEN 1 ELSE 0 END) AS worsened
FROM patient_visits
GROUP BY State
ORDER BY State;

```

Q34. Detect potential duplicate patient records.

```

SELECT Age, Gender, State, Occupation, COUNT(DISTINCT Patient_ID) AS id_count,
  STRING_AGG(DISTINCT Patient_ID, ', ') AS patient_ids
FROM patient_visits
GROUP BY Age, Gender, State, Occupation
HAVING COUNT(DISTINCT Patient_ID) > 1;

```

Section 4 — Normalization (Conceptual & Applied)

Q35. Which Normal Form does the flat patient_visits table violate, and why?

It violates **1NF**. The Symptoms column stores multiple values in a single field (e.g., "Fever, Fatigue, Increased Thirst"), and the 14 Symptom_* columns are a repeating group representing the same fact — "does this visit include symptom X?" — spread across many columns instead of being modeled as rows in a related table. 1NF requires atomic values and no repeating groups.

Q36. Why do the 14 Symptom_* binary columns violate 1NF, and how should they be restructured?

Each Symptom_* column encodes the same underlying attribute type (presence of a symptom), just for a different symptom value — this is a repeating group. Adding a new symptom would require altering the table schema (adding a column) rather than inserting a row, which is a hallmark of a 1NF violation. They should be restructured into a **symptoms** lookup table (Symptom_ID, Symptom_Name) and a bridge/junction table **visit_symptoms** (Visit_ID, Symptom_ID) that models the many-to-many relationship between visits and symptoms as rows, not columns.

Q37. Partial dependencies if (Patient_ID, Visit_Date) is used as a composite key (2NF check).

Attributes such as Age, Gender, State, Socioeconomic_Status, and Occupation depend only on Patient_ID, not on the full composite key (Patient_ID, Visit_Date) — a patient's gender doesn't change per visit date. This is a **partial dependency**, which violates **2NF** (every non-key attribute must depend on the *whole* candidate key, not just part of it). These attributes should be moved to a separate patients table keyed by Patient_ID alone.

Q38. Transitive dependencies — does BMI_Status depend on BMI_kg_meter_square? Does Glycemic_Status depend on

Fasting_Blood_Glucose__mg_dL? (3NF check)

Yes. BMI_Status (e.g., 'Overweight', 'Obesity') is derived from BMI_kg_meter_square, not directly from the visit's primary key. Similarly, Glycemic_Status ('Normal', 'Prediabetes', 'Hyperglycemia') is derived from Fasting_Blood_Glucose__mg_dL. This is a **transitive dependency**: Visit_ID → BMI_kg_meter_square → BMI_Status. It violates **3NF**, which requires non-key attributes to depend only on the key, the whole key, and nothing but the key. These derived/status columns are candidates for removal (recompute via a CASE expression or view) or, if retained for performance, should be clearly documented as derived/denormalized fields.

Q39. Proposed 3NF-compliant entity-relationship design.

Table	Primary Key	Foreign Keys
patients	Patient_ID	—
hospitals	Hospital_ID	—
diagnoses	Diagnosis_ID	—
symptoms	Symptom_ID	—
visits	Visit_ID	Patient_ID → patients, Hospital_ID → hospitals, Diagnosis_ID → diagnoses
visit_symptoms	(Visit_ID, Symptom_ID)	Visit_ID → visits, Symptom_ID → symptoms
treatments	Treatment_ID	Visit_ID → visits
imaging	Imaging_ID	Visit_ID → visits

Q40. Should Hospital_Type and State be extracted into separate reference tables?

Hospital_Type (Government/Private/Corporate) is a low-cardinality attribute of a hospital, not of a patient or visit — extracting it into a hospitals table (with a Hospital_ID surrogate key) removes redundant repeated text storage and enforces consistency (e.g., preventing typos like 'Government'). State, however, describes the **patient's residence**, not the hospital, so it correctly stays in the patients table — though it could optionally be normalized into its own states reference table if additional attributes (region, population) were needed. This follows the normalization principle of separating attributes based on which entity they truly describe.

Q41. Is full BCNF necessary, or is 3NF sufficient?

For this dataset, **3NF is generally sufficient**. BCNF resolves anomalies from overlapping composite candidate keys with functional dependencies not covered by 3NF — a scenario more relevant to tables with multiple overlapping unique constraints (e.g., a table where both (Student, Course) and (Course, Room) determine other attributes). In this healthcare schema, each table's dependencies are cleanly determined by a single-attribute or clearly-defined key, so 3NF removes the practical redundancy and update anomalies without added join complexity. Pursuing BCNF here would add design overhead for analytical reporting with limited additional integrity benefit.

Section 5 — Decomposition (Schema Design & Migration SQL)

Q42. CREATE TABLE statements for the decomposed schema.

```
CREATE TABLE patients (  
  Patient_ID    VARCHAR(20) PRIMARY KEY,  
  Age           INT,  
  Gender        VARCHAR(10),  
  State         VARCHAR(50),  
  Socioeconomic_Status VARCHAR(20),  
  Occupation    VARCHAR(100)  
);  
  
CREATE TABLE hospitals (  
  Hospital_ID  INT PRIMARY KEY AUTO_INCREMENT,  
  Hospital_Type VARCHAR(20) UNIQUE  
);  
  
CREATE TABLE diagnoses (  
  Diagnosis_ID  INT PRIMARY KEY AUTO_INCREMENT,  
  Diagnosis_Name VARCHAR(100) UNIQUE  
);  
  
CREATE TABLE symptoms (  
  Symptom_ID   INT PRIMARY KEY AUTO_INCREMENT,  
  Symptom_Name VARCHAR(100) UNIQUE  
);  
  
CREATE TABLE visits (  
  Visit_ID      INT PRIMARY KEY AUTO_INCREMENT,  
  Patient_ID    VARCHAR(20) REFERENCES patients(Patient_ID),  
  Hospital_ID   INT REFERENCES hospitals(Hospital_ID),  
  Diagnosis_ID  INT REFERENCES diagnoses(Diagnosis_ID),  
  Visit_Date    DATE,  
  Insurance_Covered VARCHAR(20),  
  Fasting_Blood_Glucose__mg_dL DECIMAL(6,1),  
  Glycemic_Status VARCHAR(20),  
  HbA1c_mmol_mol DECIMAL(6,1),  
  Total_Cholesterol_mg_dL INT,  
  BMI_kg_meter_square DECIMAL(5,1),
```

```

        BMI_Status      VARCHAR(20)
    );

CREATE TABLE visit_symptoms (
    Visit_ID INT REFERENCES visits(Visit_ID),
    Symptom_ID INT REFERENCES symptoms(Symptom_ID),
    PRIMARY KEY (Visit_ID, Symptom_ID)
);

CREATE TABLE treatments (
    Treatment_ID INT PRIMARY KEY AUTO_INCREMENT,
    Visit_ID INT REFERENCES visits(Visit_ID),
    Treatment_Type VARCHAR(50),
    Treatment_Outcome VARCHAR(20)
);

CREATE TABLE imaging (
    Imaging_ID INT PRIMARY KEY AUTO_INCREMENT,
    Visit_ID INT REFERENCES visits(Visit_ID),
    Imaging_Type VARCHAR(50),
    Imaging_Findings VARCHAR(255)
);

```

Q43. Populate the diagnoses lookup table.

```

INSERT INTO diagnoses (Diagnosis_Name)
SELECT DISTINCT Primary_Diagnosis
FROM patient_visits
WHERE Primary_Diagnosis IS NOT NULL;

```

Q44. Convert the 14 Symptom_* flag columns into rows (UNPIVOT via UNION ALL).

```

INSERT INTO visit_symptoms (Visit_ID, Symptom_ID)
SELECT v.Visit_ID, s.Symptom_ID
FROM (
    SELECT Patient_ID, Visit_Date, 'Blurred Vision' AS Symptom_Name FROM patient_visits WHERE Symptom_Blurred_Vision = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'Chest Pain' FROM patient_visits WHERE Symptom_Chest_Pain = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'Cough' FROM patient_visits WHERE Symptom_Cough = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'Fatigue' FROM patient_visits WHERE Symptom_Fatigue = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'Fever' FROM patient_visits WHERE Symptom_Fever = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'Frequent Urination' FROM patient_visits WHERE Symptom_Frequent_Urination = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'Headache' FROM patient_visits WHERE Symptom_Headache = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'High Bp' FROM patient_visits WHERE Symptom_High_Bp = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'High Fever' FROM patient_visits WHERE Symptom_High_Fever = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'Increased Thirst' FROM patient_visits WHERE Symptom_Increased_Thirst = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'Joint Pain' FROM patient_visits WHERE Symptom_Joint_Pain = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'Night Sweats' FROM patient_visits WHERE Symptom_Night_Sweats = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'Shortness Of Breath' FROM patient_visits WHERE Symptom_Shortness_Of_Breath = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'Swelling In Legs' FROM patient_visits WHERE Symptom_Swelling_In_Legs = 1
    UNION ALL
    SELECT Patient_ID, Visit_Date, 'Weight Loss' FROM patient_visits WHERE Symptom_Weight_Loss = 1
) unpivoted
JOIN visits v ON v.Patient_ID = unpivoted.Patient_ID AND v.Visit_Date = unpivoted.Visit_Date
JOIN symptoms s ON s.Symptom_Name = unpivoted.Symptom_Name;

```

Q45. Populate visits using surrogate keys from patients, hospitals, diagnoses.

```

INSERT INTO visits (
    Patient_ID, Hospital_ID, Diagnosis_ID, Visit_Date, Insurance_Covered,
    Fasting_Blood_Glucose_mg_dL, Glycemic_Status, HbA1c_mmol_mol,
    Total_Cholesterol_mg_dL, BMI_kg_meter_square, BMI_Status
)
SELECT
    pv.Patient_ID, h.Hospital_ID, d.Diagnosis_ID, pv.Visit_Date, pv.Insurance_Covered,
    pv.Fasting_Blood_Glucose_mg_dL, pv.Glycemic_Status, pv.HbA1c_mmol_mol,
    pv.Total_Cholesterol_mg_dL, pv.BMI_kg_meter_square, pv.BMI_Status
FROM patient_visits pv
JOIN hospitals h ON h.Hospital_Type = pv.Hospital_Type
JOIN diagnoses d ON d.Diagnosis_Name = pv.Primary_Diagnosis;

```

Q46. Reconstruct the original flat view (lossless-join check).

```

CREATE VIEW patient_visits_reconstructed AS
SELECT
    p.Patient_ID, p.Age, p.Gender, p.State, p.Socioeconomic_Status, p.Occupation,
    v.Visit_Date, d.Diagnosis_Name AS Primary_Diagnosis,

```

```

v.Fasting_Blood_Glucose_mg_dL, v.Glycemic_Status, v.HbA1c_mmol_mol,
v.Total_Cholesterol_mg_dL, t.Treatment_Type, t.Treatment_Outcome,
im.Imaging_Type, im.Imaging_Findings, h.Hospital_Type, v.Insurance_Covered,
v.BMI_kg_meter_square, v.BMI_Status,
STRING_AGG(s.Symptom_Name, ', ') AS Symptoms
FROM visits v
JOIN patients p      ON v.Patient_ID = p.Patient_ID
JOIN hospitals h      ON v.Hospital_ID = h.Hospital_ID
JOIN diagnoses d      ON v.Diagnosis_ID = d.Diagnosis_ID
JOIN treatments t      ON v.Visit_ID = t.Visit_ID
JOIN imaging im       ON v.Visit_ID = im.Visit_ID
JOIN visit_symptoms vs ON v.Visit_ID = vs.Visit_ID
JOIN symptoms s       ON vs.Symptom_ID = s.Symptom_ID
GROUP BY p.Patient_ID, p.Age, p.Gender, p.State, p.Socioeconomic_Status, p.Occupation,
v.Visit_Date, d.Diagnosis_Name, v.Fasting_Blood_Glucose_mg_dL, v.Glycemic_Status,
v.HbA1c_mmol_mol, v.Total_Cholesterol_mg_dL, t.Treatment_Type, t.Treatment_Outcome,
im.Imaging_Type, im.Imaging_Findings, h.Hospital_Type, v.Insurance_Covered,
v.BMI_kg_meter_square, v.BMI_Status;

-- Row-count check to confirm no data was lost or duplicated:
SELECT
  (SELECT COUNT(*) FROM patient_visits) AS original_count,
  (SELECT COUNT(*) FROM patient_visits_reconstructed) AS reconstructed_count;

```

Q47. Verify referential integrity of visit_symptoms after migration.

```

SELECT vs.Visit_ID, vs.Symptom_ID
FROM visit_symptoms vs
LEFT JOIN visits v ON vs.Visit_ID = v.Visit_ID
WHERE v.Visit_ID IS NULL;

-- An empty result confirms referential integrity (no orphaned rows).

```

Q48. Normalized vs. flat schema — trade-offs and when to denormalize.

The **normalized schema** (Section 5) is preferable for **transactional/OLTP use** — data entry, updates, and integrity enforcement — because it eliminates redundancy (e.g., a patient's Age and State are stored once, not once per visit), prevents update anomalies (changing a diagnosis name updates one row instead of thousands), and enforces valid foreign-key relationships between visits, symptoms, and treatments.

The **original flat table** is preferable for **analytical/BI reporting** — it avoids the join overhead of 8 tables when analysts run ad-hoc aggregations, and tools like Power BI, Tableau, or pandas work naturally against a single wide table.

In practice, the recommended approach is to **maintain the normalized schema as the system of record**, and then **denormalize it back into a reporting layer** — a materialized view, a nightly ETL job, or a data warehouse fact/dimension (star schema) table — so that OLTP integrity and OLAP query performance are both satisfied without compromising either.

End of document.